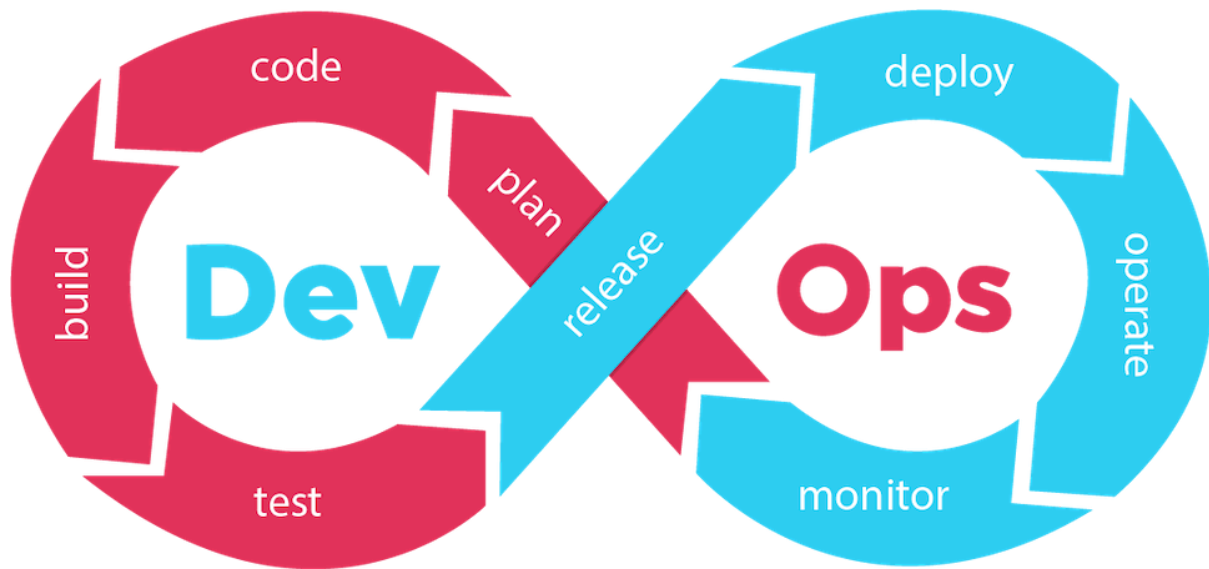




Interview Question and Answers

سوالات مصاحبه Devops قسمت دهم



kubernetes

این قسمت

Orchestration tools – init container

سوال مصاحبه « میتونی توضیح بدی InitContainer چیه و چه موقع ازش استفاده می‌کنی؟ »

حالت‌های مختلف پرسیدن سوال

● مدل ۱ – ساده و مستقیم

«میتونی توضیح بدی InitContainer چیه و چه موقع ازش استفاده می‌کنی؟»

● مدل ۲ – سناریو محور

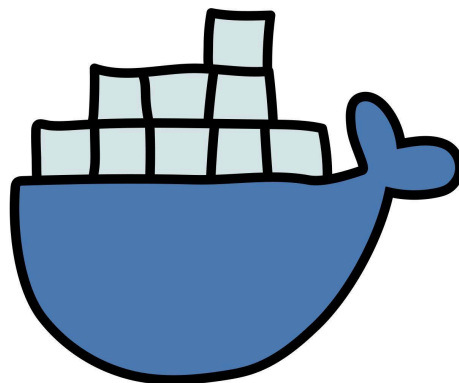
«فرض کن اپلیکیشن اصلیت برای اجرا نیاز داره که یه فایل config از یه سرویس خارجی دانلود بشه. چطوری مطمئن می‌شی قبل از استارت کانتینر اصلی اون فایل وجود داره؟»

● مدل ۳ – فنی‌تر

«تفاوت بین InitContainer و main container چیه؟ در چه شرایطی استفاده از InitContainer به جای startupProbe یا lifecycle hook منطقی‌تره؟»

● مدل ۴ – عملی و real-world

«آخرین باری که از InitContainer استفاده کردی، چه کاری باهاش انجام دادی؟ می‌تونی مثال YAML بدی؟»



مفهوم

Init Container ها کانتینرهایی هستند که قبل از اجرای کانتینرهای اصلی (main containers) در یک Pod اجرا می‌شوند.

هر پاد می‌تونه چندین InitContainer داشته باشه که به ترتیب اجرا می‌شن، و هر کدوم باید با موفقیت (exit code 0) تموم بشن تا بعدی شروع بشه.

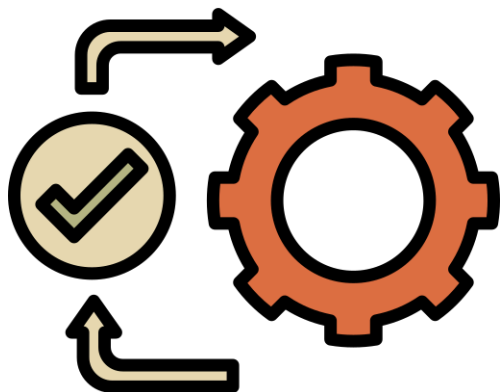
اگر حتی یکی از Init Container ها fail بشه، پاد تا زمانی که موفق نشه بالا نیاد.



سناریوهای استفاده از init container 🚀

سناریو ۱ – آماده‌سازی محیط 💡

مثلاً اگر بخواهیم قبل از شروع اپلیکیشن اصلی به دایرکتوری یا فایل خاص بسازیم یا permission درست کنیم



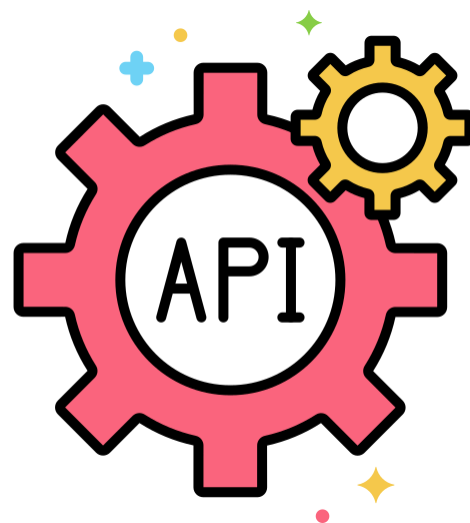
```
initContainers:  
  - name: init-permission  
    image: busybox  
    command: ["sh", "-c", "mkdir -p /data/logs  
&& chmod 777 /data/logs"]  
    volumeMounts:  
      - name: logs  
        mountPath: /data/logs
```

→ وقتی InitContainer تموم شد، کانتینر اصلی با فضای آماده‌شده بالا می‌آید.

سناریو ۲ – dependency check 💡

قبل از اجرای سرویس اصلی، می‌خواهیم مطمئن بشیم دیتابیس یا API دیگر در دسترسه

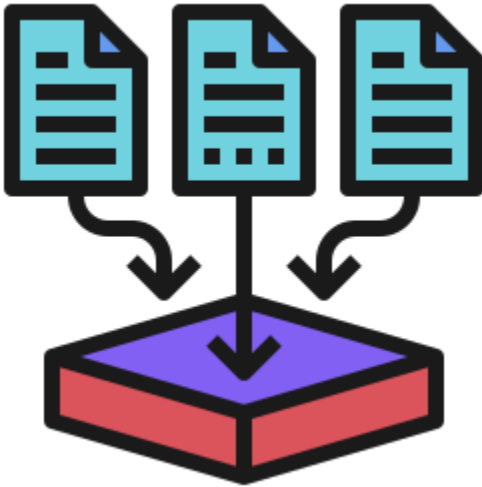
```
initContainers:  
  - name: wait-for-db  
    image: busybox  
    command: ["sh", "-c", "until nc -z  
db-service 5432; do echo waiting for db;  
sleep 2; done;"]
```



→ کانتینر اصلی فقط زمانی اجرا میشه که دیتابیس واقعاً آماده باشه.

💡 سناریو ۳ – دانلود یا pre-populate داده

مثلاً قبل از اجرای وب اپلیکیشن، باید فایل‌های static یا config از S3 یا Git گرفته بشه:



```
initContainers:
  - name: fetch-config
    image: curlimages/curl
    command: ["sh", "-c", "curl -o
/config/app.yaml https://config-server/app.yaml"]
    volumeMounts:
      - name: config-volume
        mountPath: /config
```

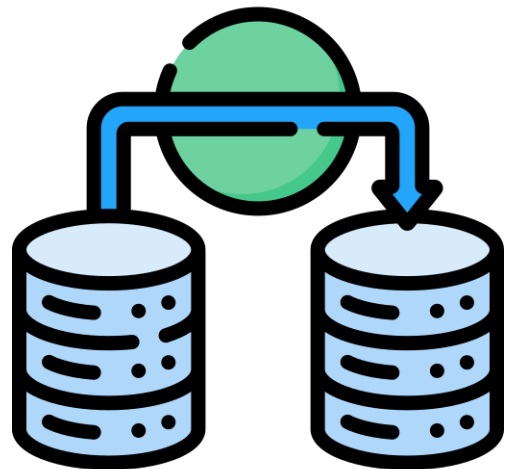
→ فایل در Volume مشترک ذخیره میشه و اپ اصلی می‌تونه ازش استفاده کنه.

💡 سناریو ۴ – اجرای migration دیتابیس

قبل از شروع backend app، schema باید آپدیت بشه:

```
initContainers:
  - name: db-migrate
    image: myapp-migrate:latest
    command: ["python", "manage.py",
"migrate"]
```

→ تضمین می‌کنه دیتابیس همیشه آپدیت و هماهنگ با ورژن اپ باشه.



🧑 جمع‌بندی پاسخ مصاحبه‌ای



من معمولاً از InitContainer زمانی استفاده می‌کنم که باید محیط اپلیکیشن اصلی رو قبل از اجرا آماده کنم – مثل آماده‌سازی دایرکتوری‌ها، تنظیم permission ها، انجام migration دیتابیس یا صبر کردن برای آماده شدن dependency هایی مثل دیتابیس یا سرویس‌های خارجی. این باعث میشه پادها تمیزتر و مقاوم‌تر بالا بیان، بدون اینکه کد اپلیکیشن اصلی با logic های آماده‌سازی آلوده بشه.



فرض کنیم شما یک اپلیکیشن Django دارید که از PostgreSQL استفاده می‌کند.
می‌خواهید قبل از بالا آمدن اپلیکیشن اصلی، دستور زیر اجرا شود تا دیتابیس migrate شود:

```
python manage.py migrate
```

برای این کار از یک **InitContainer** استفاده می‌کنیم تا قبل از اجرای وب‌سرور migration، Django انجام شود.

فایل YAML کامل (deployment + service)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: django-app
  labels:
    app: django-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: django-app
  template:
    metadata:
      labels:
        app: django-app
    spec:
      containers:
        - name: django
          image: myregistry.local/django-app:latest
          command: ["sh", "-c", "python manage.py runserver 0.0.0.0:8000"]
          env:
            - name: DATABASE_HOST
              value: postgres-service
            - name: DATABASE_NAME
              value: mydb
            - name: DATABASE_USER
              value: myuser
            - name: DATABASE_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-secret
                  key: password
      ports:
        - containerPort: 8000
      volumeMounts:
        - name: app-volume
          mountPath: /app
```

```
initContainers:
  - name: db-migrate
    image: myregistry.local/django-app:latest
    command: ["sh", "-c", "python manage.py migrate"]
    env:
      - name: DATABASE_HOST
        value: postgres-service
      - name: DATABASE_NAME
        value: mydb
      - name: DATABASE_USER
        value: myuser
      - name: DATABASE_PASSWORD
        valueFrom:
          secretKeyRef:
            name: db-secret
            key: password
    volumeMounts:
      - name: app-volume
        mountPath: /app
volumes:
  - name: app-volume
    emptyDir: {}

---
apiVersion: v1
kind: Service
metadata:
  name: django-service
spec:
  selector:
    app: django-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8000
```